

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет

Курсовая работа: Разработка прототипа мессенджера

Выполнила:

Едигарева Дарья

Группа К3439

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1. Введение.....	3
2. Цели и задачи работы.....	4
3. Описание архитектуры приложения.....	5
4. Реализация функциональности по этапам лабораторных работ.....	9
4.1 Этап 1: Каркас приложения и навигация.....	9
4.2 Этап 2: Состояние ViewModel и динамика данных.....	11
4.3 Этап 3: Сетевое взаимодействие и локальное хранение.....	15
4.4 Этап 4: Улучшение UI, интерактивность и фоновая работа.....	23
5. Выводы.....	32

1. Введение

Общая идея и цель разработки: Курсовой проект представляет собой Android-приложение «Messenger» – упрощенный мессенджер, разработанный на Kotlin. Цель проекта – отработать современные подходы к архитектуре Android-приложений и освоить практики построения устойчивого и масштабируемого кода. Приложение состоит из одной главной активности и нескольких экранов (фрагментов), между которыми реализована навигация. Пользовательский интерфейс включает ленту сообщений, экран профиля и настройки. Несмотря на название, приложение служит учебным примером и не выполняет полный функционал обмена сообщениями, вместо этого оно загружает список сообщений из сетевого API и отображает их в виде ленты.

В проекте основной упор сделан на разделение ответственностей по шаблону Model-View-ViewModel (MVVM). Это обеспечивает ясную структуру и легкость расширения приложения. В ходе работы реализованы следующие аспекты:

- Навигация и структура UI: Одна Activity выступает контейнером с Bottom Navigation для переключения между экранами (лента, профиль, настройки).
- Управление состоянием и архитектура MVVM: Используются ViewModel и LiveData для хранения и наблюдения за данными, обеспечивая сохранение состояния при поворотах экрана и разделение логики.
- Работа с сетью и локальным хранением: Данные сообщений загружаются через Retrofit из внешнего API и кэшируются в локальной базе данных Room для офлайн-доступа.
- Фоновая синхронизация и уведомления: Настроен WorkManager для периодического обновления данных в фоне, при поступлении новых данных отображаются системные уведомления.
- Material Design и интерактивность: Интерфейс улучшен с помощью компонентов Material (CardView, FloatingActionButton, AppBarLayout и др.), реализована возможность лайкать сообщения.

2. Цели и задачи работы

Цель работы: разработать и продемонстрировать прототип мессенджера с архитектурой MVVM, устойчивым хранением состояния и поддержкой сетевого взаимодействия и фоновых задач. Приложение должно быть построено с четким разделением ответственности, согласно принципам профессиональной разработки Android-приложений.

Задачи проекта: для достижения указанной цели последовательно решены следующие задачи:

- Создание каркаса приложения: реализовать Single-Activity структуру с Bottom Navigation, где одна Activity содержит NavHost для трех фрагментов (лента, профиль, настройки). Обеспечить навигацию с помощью Jetpack Navigation и добавить базовые UI-элементы (на экране ленты – заглушка текста, на экране профиля – отображение данных пользователя, на экране настроек – переключатель темы). Настроить вывод сообщений в лог для отслеживания жизненного цикла Activity/Fragment.
- Реализация хранения состояния: интегрировать ViewModel для экранов профиля и настроек, хранящий пользовательские данные (имя, статус, выбор темы). Обеспечить привязку UI к данным через LiveData и тем самым сохранение и восстановление состояния при повороте экрана. Логи дополняются сообщениями о создании и уничтожении ViewModel.
- Работа с сетью и БД: разработать слой данных: создать Retrofit-клиент для загрузки списка сообщений с публичного API и Room-базу данных для кеширования этих сообщений. Реализовать Repository как единую точку доступа к данным для ViewModel. Функционал: при запуске загрузить список сообщений через сеть и отобразить в ленте с помощью RecyclerView; при отсутствии интернета – подгружать данные из локальной БД; добавить кнопку «Обновить» для ручной синхронизации данных.
- UI-улучшения и фоновая работа: улучшить интерфейс, применив компоненты Material Design (например, карточки сообщений через CardView, кнопка действий – FloatingActionButton, верхняя панель – AppBarLayout). Добавить интерактивность: возможность ставить «лайк» на сообщение (иконка меняется при нажатии). Настроить

периодическую фоновую синхронизацию списка сообщений с сервером с помощью WorkManager (обновление раз в 15 минут). При успешной синхронизации в фоне отображать Notification с текстом «Новые данные получены». При запуске запрашивать у пользователя необходимые runtime-разрешения (на отправку уведомлений для Android 13+, доступ к контактам) и учитывать состояние сети (отображать индикатор офлайн-режима, отключать кнопку обновления без интернета).

Все перечисленные задачи были выполнены последовательно. В итоге получено приложение с чистой архитектурой, где каждый компонент выполняет строго свою роль. Ниже приводятся технические детали реализации по разделам архитектуры и этапам разработки.

3. Описание архитектуры приложения

Приложение построено по архитектурному шаблону MVVM (Model – View – ViewModel) с использованием репозитория и реактивных данных. Такая архитектура позволяет разделить логику приложения на слои, снизить связность компонентов и упростить поддержку кода. Ниже описаны основные слои и компоненты архитектуры:

- View (Представление): отвечает за отображение данных и взаимодействие с пользователем. В приложении роль View выполняют Activity (главное окно с контейнером для фрагментов) и набор Fragment-ов для отдельных экранов. Представление содержит UI-логику (настройка виджетов, обработка кликов), но не содержит бизнес-логики или данных – при необходимости получить или изменить данные View обращается к ViewModel. В нашем случае MainActivity и фрагменты делегируют всю работу с данными соответствующим ViewModel. Например, Fragment «Профиль» обновляет UI при изменениях данных профиля и передает действия (например, ввод текста) во ViewModel.
- ViewModel: посредник между View и моделью данных. Хранит состояние UI и реализует логику, связанную с экраном (например, реакция на действия пользователя), но не знает о том, откуда приходят данные. ViewModel получает данные только через репозиторий (абстракцию модели) и предоставляет эти данные View через наблюдаемые объекты (LiveData). Также ViewModel

принимает события от View – например, вызовы методов при пользовательском вводе – и обновляет состояние. В жизненном цикле приложение ViewModel живет дольше соответствующего фрагмента/активности и переживает повороты экрана, сохраняя данные. При окончательном уничтожении UI (например, закрытии экрана) ViewModel также уничтожается, что фиксируется в `onCleared()`.

- **Model (модель данных) и Repository:** В контексте Android MVVM под моделью подразумевается совокупность слоев работы с данными: сетевые источники, база данных, бизнес-логика. В проекте реализован слой Repository – класс, инкапсулирующий доступ к данным из разных источников (удаленного API и локальной базы). Репозиторий предоставляет ViewModel чистый интерфейс (методы) для получения или обновления данных и сам решает, откуда получать данные – из сети или кеша. Такой подход отделяет ViewModel от деталей реализации API или SQL – вместо этого она работает с абстракцией репозитория. Принцип единственного источника данных соблюдается: репозиторий выступает Single Source of Truth для списка сообщений – он обновляет локальную БД и выступает источником достоверных данных для UI.
- **Локальная база данных (Room):** используется для хранения сообщений офлайн. Реализован класс базы данных (наследник `RoomDatabase`) с определением единственной таблицы `messages` и DAO-интерфейса доступа. DAO (`Data Access Object`) содержит методы для запросов: выбор всех сообщений и вставка/обновление списка сообщений. Room обеспечивает наблюдение за изменениями данных: DAO метод, возвращающий `LiveData`, сразу нотифицирует подписчиков (ViewModel/UI) при изменении таблицы. Это реализует реактивный поток данных: при записи новых сообщений в БД интерфейс автоматически обновляется без дополнительного кода.
- **Удаленный источник (API):** для получения данных сообщений использован открытый API сервиса `JSONPlaceholder` (эндпоинт `/posts`). Используется библиотека `Retrofit` с конвертером `Gson` для выполнения HTTP-запросов в фоновом потоке (через корутины). Определен интерфейс `MessageApi` с методом `getMessages()`, возвращающим список объектов `MessageDto`. Репозиторий вызывает этот метод при необходимости обновить данные.

- LiveData и реактивность UI: Связь между ViewModel и View реализована с помощью LiveData. ViewModel хранит данные в MutableLiveData, а UI-фрагменты подписываются на эти объекты как LiveData. При изменении данных (например, загружены новые сообщения или пользователь отредактировал поле) LiveData автоматически уведомляет View, который обновляет соответствующие элементы экрана. Благодаря этому достигается односторонний поток данных (unidirectional data flow): данные текут от репозитория через ViewModel к UI, а события пользователя текут от UI к ViewModel, где обрабатываются. Такой поток упрощает отслеживание изменений и предотвращает непредсказуемое поведение.
- Навигация: Приложение использует Jetpack Navigation Component для переключения экранов во фрагментах. Навигационный граф (nav_graph.xml) описывает три основные Destination (Feed, Profile, Settings). MainActivity содержит контейнер NavHostFragment. Переключение вкладок BottomNavigationView синхронизировано с NavController. Данный подход упрощает управление Back Stack и обеспечивает согласованность навигации.
- Фоновая работа: Для периодической синхронизации данных используется WorkManager. В классе Application при старте планируется уникальная периодическая задача (PeriodicWorkRequest) с интервалом ~15 минут, требующая подключения к сети. Эта задача выполняется даже если пользователь не взаимодействует с приложением, и по расписанию вызывает обновление сообщений через репозиторий. По окончании фоновой синхронизации формируется уведомление для пользователя.
- Обработка системных событий: Приложение учитывает состояние сети. В момент обновления ленты (по кнопке или в фоне) проверяется доступность интернета. Если сеть недоступна, UI показывает индикатор офлайн-режима (текст «Офлайн: показываю данные из базы») и отключает кнопку обновления. При появлении подключения функциональность восстанавливается. Кроме того, для показа уведомлений на Android 13+ запрашивается разрешение POST_NOTIFICATIONS у пользователя при первом запуске. Разрешение на чтение контактов (READ_CONTACTS) также

заявлено (в перспективе, например, для получения списка контактов), но в текущем функционале не задействовано.

Направленность зависимостей: Архитектура спроектирована так, что зависимости направлены сверху вниз – UI знает о ViewModel, ViewModel знает о репозитории, репозиторий знает о конкретных источниках данных (API, DAO). Обратные зависимости запрещены: ни репозиторий, ни тем более база не обращаются к UI, и ViewModel не хранит ссылок на View. Это соответствует принципам чистой архитектуры и обеспечивает модульность – например, реализацию хранилища или API можно заменить, не изменяя логику ViewModel и UI. Такой подход формирует предсказуемый поток данных и устойчивое состояние приложения. Как следствие, приложение легко тестировать (каждый слой независим) и расширять новыми функциями без риска нарушить существующий код.

Ниже представлена схематическая диаграмма архитектуры и описание реализации ключевых компонентов на примере нашего приложения.

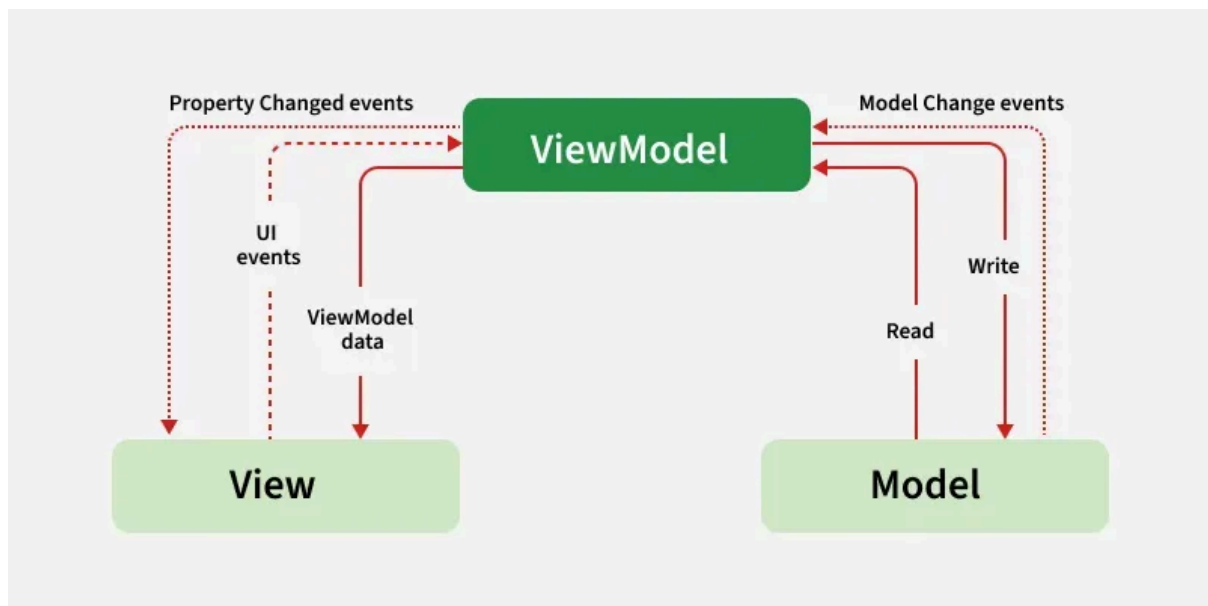


Рисунок 1. Архитектура приложения (слои MVVM)

4. Реализация функциональности по этапам лабораторных работ

4.1 Этап 1: Каркас приложения и навигация

Структура Activity и Fragments: На первом этапе создано базовое приложение с одной Activity и тремя фрагментами. MainActivity выступает

контейнером для фрагментов и содержит BottomNavigationView для переключения экранов. Был настроен Navigation Graph с тремя фрагментами: FeedFragment (лента), ProfileFragment (профиль пользователя) и SettingsFragment (настройки). Навигация выполнена стандартными средствами: NavHostFragment в разметке MainActivity и NavController для управления переходами. BottomNavigationView связывается с NavController, чтобы при выборе пункта меню отображался соответствующий фрагмент. Ниже приведен фрагмент кода из MainActivity.kt, показывающий инициализацию навигации:

// Инициализация Navigation Component в MainActivity

```
val navHostFragment = supportFragmentManager
    .findFragmentById(R.id.nav_host) as
NavHostFragment
val navController = navHostFragment.navController
binding.bottomNav.setupWithNavController(navController)
```

В коде выше R.id.nav_host – контейнер (FragmentContainerView) в макете активности, где будут отображаться фрагменты. Метод setupWithNavController автоматически синхронизирует нажатия на пункты BottomNavigation с соответствующими переходами NavController. Таким образом, структура приложения отвечает требованию одной Activity с внутренней навигацией.

Экраны приложения: На данном этапе экраны имели минимальный функционал:

- Лента (FeedFragment): пока что заглушка – простой TextView с надписью, например, «Здесь будет лента сообщений».
- Профиль (ProfileFragment): отображает несколько статичных элементов (например, поле e-mail) и может содержать EditText для имени пользователя или статуса, однако сохранение введенных данных на этом шаге еще не реализовано (оно появится во втором этапе).
- Настройки (SettingsFragment): содержит переключатель (Switch) для выбора темы (светлая/темная). Пока что переключатель влияет напрямую на AppCompatActivity и переключает тему приложения,

но состояние не сохраняется без ViewModel (реализовано на этапе 2).

Жизненный цикл и логирование: Для демонстрации понимания жизненного цикла Android-компонентов, в методах жизненного цикла Activity и Fragment (onCreate, onStart, onResume, onPause, onStop, onDestroy) были добавлены вызовы Log.d(...). При запуске и закрытии приложения, а также при навигации между фрагментами, логи фиксируют эти события. Например, в MainActivity.onStart() выводится Log.d(TAG, "onStart"). Аналогично, в каждом фрагменте переопределены onCreate/onDestroyView для логирования.

Итог по этапу 1: Приложение успешно запускалось и переключало экраны через нижнюю навигацию. Код был структурирован с выделением отдельных классов под каждый экран. Этот фундамент позволил приступить к внедрению архитектуры MVVM и хранению состояния на следующем этапе.

4.2 Этап 2: Состояние ViewModel и динамика данных

На втором этапе был внедрен архитектурный паттерн MVVM для управления состоянием экрана Профиля и Настроек, а также сохранения данных при повороте экрана. Основная идея – переместить данные UI (текстовые поля профиля, состояние переключателя темы) из фрагментов во ViewModel, чтобы они пережили пересоздание UI-компонентов.

ViewModel для экрана профиля: Создан класс ProfileScreenViewModel, наследующий ViewModel. Он хранит два поля: имя пользователя (nickname) и текущий статус. Эти поля определены как MutableLiveData, а наружу экспонируются как LiveData для наблюдения:

```
package com.example.messengerlab1.ui.profile

import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class ProfileScreenViewModel : ViewModel() {
```

```

private val tag = "ProfileScreenVM"
private val _nickname = MutableLiveData("Дарья")
val nickname: LiveData<String> = _nickname
private val _mood = MutableLiveData("Пытаюсь
разобраться с MVVM")
val mood: LiveData<String> = _mood

init {
    Log.d(tag, "init: ViewModel профиля создана")
}

fun onNicknameChanged(newName: String) {
    Log.d(tag, "onNicknameChanged: $newName")
    _nickname.value = newName
}

fun onMoodChanged(newMood: String) {
    Log.d(tag, "onMoodChanged: $newMood")
    _mood.value = newMood
}

override fun onCleared() {
    Log.d(tag, "onCleared: ViewModel профиля
будет уничтожена")
    super.onCleared()
}
}

```

В коде выше видно, что ViewModel инициализирует поля значениями по умолчанию (имя «Дарья» и пример статуса). Метод onNicknameChanged вызывается из UI, присваивая новое значение в _nickname – благодаря LiveData это автоматически сообщит наблюдателям об изменении. Метод onCleared() выводит лог при уничтожении ViewModel, что происходит только когда экран полностью закрывается (например, при завершении активности). ViewModel для экрана настроек создан аналогично.

SettingsScreenViewModel с полем `_isNightMode`: `MutableLiveData<Boolean>` для хранения выбранной темы (светлая или темная). Этот ViewModel содержит метод `setNightMode(enabled: Boolean)` для обновления состояния. При инициализации переключателя Switch на экране настроек, UI будет наблюдать за `isNightMode LiveData` и применять соответствующую тему приложения:

```
package com.example.messengerlab1.ui.settings

import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class SettingsScreenViewModel : ViewModel() {

    private val tag = "SettingsScreenVM"
    private val _isNightMode = MutableLiveData(false)
    val isNightMode: LiveData<Boolean> = _isNightMode

    init {
        Log.d(tag, "init: ViewModel настроек  
создана")
    }

    fun setNightMode(enabled: Boolean) {
        Log.d(tag, "setNightMode: $enabled")
        _isNightMode.value = enabled
    }

    override fun onCleared() {
        Log.d(tag, "onCleared: ViewModel настроек  
будет уничтожена")
        super.onCleared()
    }
}
```

Первоначальное значение false означает светлую тему по умолчанию. В SettingsFragment мы связываем переключатель с этим состоянием.

Привязка ViewModel к UI: Фрагменты профиля и настроек получают экземпляры своих ViewModel с помощью делегата by viewModels(). Это автоматически обеспечивает, что при повторном создании фрагмента (например, повороте экрана) будет возвращен тот же объект ViewModel (пока существует Activity), а не новый. В ProfileFragment после создания View выполняется настройка наблюдателей LiveData:

В ProfileFragment в onCreateViewCreated:

```
viewModel.nickname.observe(viewLifecycleOwner) { nick
->
    if (binding.inputNickname.text.toString() !=
nick) {
        binding.inputNickname.setText(nick)
    }
}

viewModel.mood.observe(viewLifecycleOwner) { text ->
    if (binding.inputMood.text.toString() != text) {
        binding.inputMood.setText(text)
    }
}

binding.inputNickname.addTextChangedListener {
    editable ->

viewModel.onNicknameChanged(editable?.toString().orEmpty())
}

binding.inputMood.addTextChangedListener { editable
->

viewModel.onMoodChanged(editable?.toString().orEmpty()
())
```

```
}
```

Здесь первое наблюдение следит за nickname: когда LiveData изменится (или при присоединении сразу отдаст текущее значение), мы обновляем EditText поля ввода имени, если оно отличается. Аналогично для mood (поле «настроение/статус»). Такая проверка `if != text` нужна, чтобы избежать циклических обновлений (когда мы сами установили текст, не триггерить лишний вызов). Далее, с помощью `addTextChangedListener` на каждом поле мы передаем изменения обратно в ViewModel: при вводе нового имени вызывается `onNicknameChanged`. Получается двусторонняя связь: ViewModel - UI через LiveData, и UI - ViewModel через вызовы методов. Это реализует паттерн реактивного обновления интерфейса: любое изменение данных сразу отражается на экране, и наоборот, пользовательский ввод моментально сохраняется в состоянии ViewModel.

В SettingsFragment похожим образом сделано для переключателя темы:

- Наблюдение `viewModel.isNightMode.observe` переключает тему через `AppCompatActivity.setDefaultNightMode(...)` и устанавливает положение Switch.
- Слушатель `switchTheme.setOnCheckedChangeListener` вызывает `viewModel.setNightMode(isChecked)` при изменении пользователем.
- Таким образом, состояние темы хранится во ViewModel, и при повороте экрана или возвращении на экран настроек переключатель сохранит правильное положение, а тема останется консистентной.

Сохранение состояния при повороте: Благодаря использованию ViewModel + LiveData, данные, введенные на экране профиля, не теряются при смене конфигурации (например, повороте устройства). Ранее, без ViewModel, при повороте Activity уничтожалась и создавалась заново, и EditText возвращались к дефолтным значениям. Теперь же ViewModel живет в пределах жизненного цикла Activity и переживает пересоздание фрагментов. По возвращении UI сразу подтягивает текущее значение LiveData (которое сохранилось). Это демонстрирует одно из ключевых преимуществ ViewModel – кэширование состояния и переживание конфигурационных изменений.

Логирование ViewModel: В конструкторе (init { ... }) и методе onCleared() каждого ViewModel выведены сообщения Log.d. Например, при открытии профиля лог покажет ProfileScreenVM: init: ViewModel профиля создана, а при закрытии приложения – ... будет уничтожена. Это подтверждает, что ViewModel создается единожды и уничтожается не при каждом повороте, а только при завершении работы.

Итог по этапу 2: Экран «Профиль» теперь позволяет вводить имя и статус, и эти данные сохраняются во ViewModel – при возврате на экран или повороте устройства введенный текст остается. Экран «Настройки» корректно переключает тему и помнит выбор. Реализовано реактивное обновление UI через LiveData: если изменить данные во ViewModel программно, UI обновится автоматически. Таким образом, приложение приобрело устойчивое состояние и четкое разделение ответственности: View отображает и отправляет события, ViewModel хранит и изменяет данные, не зная о подробностях UI.

4.3 Этап 3: Сетевое взаимодействие и локальное хранение

Третий этап добавил в приложение реальную функциональность загрузки и хранения сообщений, выполнив тем самым переход от статических данных к динамическим. Основные реализации этого этапа:

- Сетевой клиент Retrofit для получения списка сообщений.
- Локальная база данных Room для хранения сообщений офлайн.
- Repository (MessageRepository), объединяющий логику получения/сохранения данных.
- Отображение списка сообщений в ленте (RecyclerView), включая ручное обновление и офлайн-режим.

Модель данных сообщений: Во избежание прямой зависимости UI от формата API, были созданы модельные классы:

- MessageDto – data class для объектов из API (поля id, title, body).
- MessageEntity – @Entity класс для Room, соответствующий таблице messages. В нашем случае структура практически совпадает: id (ключ), title и body.

Retrofit API: Интерфейс MessageApi объявлен с методом:

```
interface MessageApi {  
    @GET("posts")  
    suspend fun getMessages(): List<MessageDto>  
}
```

Он обращается к эндпоинту «posts» сервиса JSONPlaceholder и возвращает список сообщений. Настроен singleton RetrofitClient с baseUrl (<https://jsonplaceholder.typicode.com/>) и Gson конвертером. В компоненте Application (на этапе запуска приложения) создается экземпляр API:

```
val api: MessageApi by lazy {  
    Retrofit.Builder()  
        .baseUrl(BASE_URL)  
  
    .addConverterFactory(GsonConverterFactory.create())  
        .build()  
        .create(MessageApi::class.java)  
}
```

Этот API будет использоваться репозиторием для сетевых запросов. Все вызовы определены как suspend (используют корутины), чтобы их можно было выполнять в фоновом потоке.

База данных Room: Определен класс AppDatabase с аннотацией @Database(entities=[MessageEntity::class], version=1). В нем описан единственный DAO:

```
@Dao  
interface MessageDao {  
  
    @Query("SELECT * FROM messages ORDER BY id DESC")  
    fun observeAll(): LiveData<List<MessageEntity>>  
  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun upsertAll(items: List<MessageEntity>)  
}
```


Как видно, `observeAll()` возвращает `LiveData` списка сообщений, отсортированных по `id` по убыванию (то есть свежие сообщения первыми). `Room` автоматически генерирует реализацию этого DAO. Метод возвращает `LiveData`, поэтому при подписке `ViewModel` сразу получит актуальные данные, а при изменении таблицы `Room` уведомит всех подписчиков (реактивное поведение). Метод `upsertAll` вставляет список сообщений, при конфликте по первичному ключу выполняя замену (то есть обновляет записи, если такие `id` уже есть). База данных создается один раз в классе `Application` при старте и доступна через синглтон `App.database`:

```
database = Room.databaseBuilder(  
    applicationContext,  
    AppDatabase::class.java,  
    "messenger_db"  
) .build()
```

Репозиторий `MessageRepository`: Этот класс сводит воедино логику получения данных. Его задача – по запросу выдать `ViewModel` списки сообщений, управляя источниками данных (сеть/база). Репозиторий в конструкторе принимает `MessageApi` и `MessageDao`. Основные функции:

- `fun observeMessages(): LiveData<List<MessageEntity>>` – просто делегирует вызов к `dao.observeAll()`. `ViewModel` ленты будет использовать этот метод, чтобы наблюдать за данными из базы (через репозиторий, не напрямую). Репозиторий не обрабатывает `LiveData`, он сразу возвращает `LiveData` из DAO, выступая прозрачным посредником.
- `suspend fun refresh()` – осуществляет обновление данных из сети и сохранение в базу:

```
class MessageRepository(  
    private val api: MessageApi,  
    private val dao: MessageDao  
) {  
    private val tag = "MessageRepository"  
  
    fun observeMessages() = dao.observeAll()
```

```

suspend fun refresh() {
    Log.d(tag, "refresh: start")
    val remote = api.getMessage()
    val entities = remote.map { MessageEntity(id
= it.id, title = it.title, body = it.body) }
    dao.upsertAll(entities)
    Log.d(tag, "refresh: saved ${entities.size}
items")
}
}

```

Здесь сначала выполняется сетевой запрос `getMessage()` (при отсутствии интернета этот вызов бросит исключение, которое мы обрабатываем выше по стеку). Затем результат (список `MessageDto`) конвертируется в список сущностей базы (`MessageEntity`). Далее вызывается `dao.upsertAll`, что записывает все записи в базу. Благодаря Room, сразу после вставки, активное `LiveData<List<MessageEntity>>` (которое, возможно, уже наблюдается во `ViewModel`) получит уведомление с обновленным списком данных. То есть после выполнения `refresh()` UI автоматически отобразит новые данные. Этот механизм – основа принципа офлайн-кэша: после единоразовой загрузки из сети, все отображение идет из локальной БД, а сеть только обновляет содержимое базы.

Все сетевые операции выполняются через корутины на IO-поток (благодаря Retrofit и coroutine support). Вызов `refresh()` мы будем запускать из `ViewModel` внутри `viewModelScope.launch { ... }`, что обеспечивает выполнение вне основного потока.

FeedViewModel и отображение ленты: Для экрана «Лента» создан `FeedViewModel`, принимающий репозиторий (через фабрику, поскольку нужен параметр). Он получает `LiveData` списка сообщений из репозитория и предоставляет его UI, а также реализует логику обновления:

```

class FeedViewModel(
    private val repository: MessageRepository
) : ViewModel() {

    private val tag = "FeedVM"

```

```

        val messages: LiveData<List<MessageEntity>> =
repository.observeMessages()

        init {
            Log.d(tag, "init: created")
        }

        private val _syncSuccess =
MutableLiveData<Unit>()
        val syncSuccess: LiveData<Unit> = _syncSuccess

        fun refresh() {
            Log.d(tag, "refresh: clicked")
            viewModelScope.launch {
                try {
                    repository.refresh()
                    _syncSuccess.postValue(Unit)
                    Log.d(tag, "refresh: success")

                } catch (e: Exception) {
                    Log.d(tag, "refresh: failed ->
${e.message}")
                }
            }
        }

        override fun onCleared() {
            Log.d(tag, "onCleared: destroyed")
            super.onCleared()
        }
    }
}

```

Здесь messages подписан напрямую на DAO через репозиторий – то есть при изменении записей в базе (в результате refresh() или при старте) эта LiveData предоставляет фрагменту актуальный список. Метод refresh() запускает repository.refresh() в фоновом потоке. В случае успеха, мы публикуем событие _syncSuccess (типа Unit) – оно используется, чтобы уведомить UI о том, что синхронизация прошла успешно и можно, например, показать уведомление. В случае ошибки (исключение, например

отсутствие сети), мы просто логируем – UI на уровне ленты дополнительно обрабатывает видимость кнопки через проверку сети.

Интеграция в FeedFragment: Во FeedFragment.onCreate() мы получаем экземпляр FeedViewModel через фабрику, используя глобальный Application для доступа к репозиторию:

```
val app = requireActivity().application as App
val factory =
FeedViewModelFactory(app.messageRepository)
viewModel = ViewModelProvider(this,
factory) [FeedViewModel::class.java]
```

Далее, в onCreateView настраиваем RecyclerView:

```
binding.rvMessages.layoutManager =
LinearLayoutManager(requireContext())
binding.rvMessages.adapter = adapter

viewModel.messages.observe(viewLifecycleOwner) {
items ->
    Log.d(tagLog, "messages observed: ${items.size}")
    adapter.submitList(items)
}
```

Таким образом, как только в БД появятся сообщения, адаптер получит список и отобразит элементы в RecyclerView. В начале работы (при первом открытии экрана) мы можем вызвать viewModel.refresh() вручную, чтобы инициировать загрузку из сети. Также настроена кнопка обновления:

```
binding.fabRefresh.setOnClickListener {
    val onlineNow = isOnline(requireContext())
    binding.tvOffline.visibility = if (onlineNow)
View.GONE else View.VISIBLE
    binding.fabRefresh.isEnabled = onlineNow

    if (onlineNow) viewModel.refresh()
}
```

Метод `isOnline()` проверяет через `ConnectivityManager`, есть ли сейчас подключение к интернету. Если сети нет, мы не вызываем `refresh`, а отображаем `tvOffline` (специальный `TextView` с текстом «Оффлайн: показываю данные из базы») и делаем `fabRefresh` неактивной. Если сеть есть – запускаем `refresh` и скрываем индикатор оффлайна. Эта логика выполнена, чтобы при ручном обновлении не пытаться дергать API без интернета, а вместо этого явно показать пользователю, что он в оффлайне.

При успешном завершении `viewModel.refresh()`, `FeedViewModel` устанавливает `_syncSuccess`, на которое фрагмент подписан:

```
viewModel.syncSuccess.observe(viewLifecycleOwner) {  
  
    com.example.messengerlab1.work.notify.SyncNotifier.s  
how(  
    requireContext(),  
    "Новые данные получены"  
)  
}
```

Здесь `SyncNotifier` – вспомогательный объект, который формирует системное уведомление. Таким образом, даже при ручном обновлении (кнопкой) пользователь сразу видит уведомление о результате.

RecyclerView Адаптер: Пока что можно использовать простой `ArrayAdapter` или `ListAdapter` для отображения сообщений. Мы реализовали класс `MessagesAdapter` (унаследованный от `ListAdapter<MessageEntity, ...>`), привязанный к макету `item_message.xml`.

Оффлайн-доступ: Благодаря тому, что `FeedViewModel.messages` берет данные из `Room` (через `LiveData`), приложение автоматически поддерживает офлайн-режим. Алгоритм:

- При старте `FeedFragment` вызывается `viewModel.refresh()`. Если есть интернет, удастся загрузить и сохранить сообщения, и UI их показывает. Если интернета нет (например, приложение запущено офлайн), вызов `repository.refresh()` выбросит исключение и ничего не сохранит.
- Но даже если `refresh` не отработал, `viewModel.messages.observe` все равно активен. Если в базе остались данные от предыдущих

запусков, они сразу отобразятся (Room дает последний закешированный список). Если база пуста (первый запуск офлайн) – список будет пустым, лента останется пустой, но офлайн-баннер уведомит пользователя.

- Таким образом, при отсутствии сети приложение показывает данные из локальной базы (если они есть). А при появлении сети пользователь может нажать обновление, и новые данные будут скачаны и сохранены.

Итог по этапу 3: На этом шаге приложение превратилось в работающий прототип: экран ленты теперь загружает реальные данные (100 постов от API), сохраняет их и отображает. Было выполнено требование интеграции сети и локального хранилища через единый репозиторий. Логика обновления полностью вынесена из UI: фрагмент ленты просто вызывает `viewModel.refresh()`, не зная, что там сеть или база – этим управляет репозиторий (принцип абстракции источников данных соблюден). Приложение корректно работает без интернета, демонстрируя последние сохраненные данные офлайн. Пользователь может вручную обновить список.

Такая архитектура соответствует рекомендациям: `ViewModel` не знает о том, откуда приходят данные (API или БД), он обращается к репозиторию, а репозиторий и `Room` вместе обеспечивают реактивный поток данных к UI. Мы получили пример «ленивой» загрузки: UI всегда отображает `LiveData` из базы, а обновление базы происходит асинхронно. Реализация задач этапа доказала удобство MVVM: большая часть кода UI осталась простой (получить `ViewModel` и подписаться на `LiveData`), а вся сложность – в репозитории и слое данных.

4.4 Этап 4: Улучшение UI, интерактивность и фоновая работа

На заключительном этапе проект получил улучшения интерфейса и дополнительные функции: лайки, уведомления, фоновые обновления данных и прочие улучшения по заданию.

UI и Material Design: Вёрстка экрана ленты была изменена для улучшения внешнего вида:

- Каждое сообщение отображается в карточке (Material CardView) с закругленными углами и тенью. Внутри карточки – аватар пользователя (пока статическое изображение, например, иконка приложения), имя (заголовок сообщения) и текст (тело сообщения).
- Добавлена кнопка «Лайк» (сердечко) в углу карточки, позволяющая отметить понравившееся сообщение.
- Использован FloatingActionButton (FAB) для кнопки обновления. FAB с иконкой обновления (ic_popup_sync) размещён поверх списка (с помощью CoordinatorLayout) в правом нижнем углу (anchor к RecyclerView/CardView), что лучше соответствует Material Design, чем обычная Button.
- В верхней части ленты есть AppBarLayout/Toolbar с заголовком приложения. В комбинации с CoordinatorLayout это даёт материал-дизайн скроллинга.
- Экран профиля обернут в ScrollView для прокрутки, содержимое центрировано. Также для визуального оформления можно было добавить разделители, иконки или другое – но основное требование Material Design касалось ленты.

Во фрагменте ниже приведена выдержка из layout fragment_feed.xml, демонстрирующая использование Material-компонентов: CardView для списка и FloatingActionButton:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">
```

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"

    android:layout_height="?attr/actionBarSize"
    android:title="Лента" />

</com.google.android.material.appbar.AppBarLayout>
```

```
    <TextView
        android:id="@+id/tvOffline"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Оффлайн: показываю данные из
базы"

        android:padding="8dp"
        android:visibility="gone" />
```

```
    <androidx.cardview.widget.CardView
        android:id="@+id/cardFeed"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_margin="16dp"
        app:cardCornerRadius="16dp"
        app:cardUseCompatPadding="true"

        app:layout_behavior="@string/appbar_scrolling_view_b
ehavior">
```

```
        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/rvMessages"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="8dp" />

    </androidx.cardview.widget.CardView>
```



```

<com.google.android.material.floatingactionbutton.Fl
oatingActionButton
    android:id="@+id/fabRefresh"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:contentDescription="Обновить"
    app:layout_anchor="@id/cardFeed"
    app:layout_anchorGravity="bottom|end"

app:srcCompat="@android:drawable/ic_popup_sync" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Карточка cardFeed содержит RecyclerView с отступами. FAB привязана (layout_anchor) к этой CardView и расположена внизу справа. TextView tvOffline находится над списком и скрыт (visibility="gone"), появляется, если приложение офлайн. Таким образом, требование применить CardView, FAB, AppBarLayout выполнено: UI ленты выглядит как современный список сообщений.

Адаптер сообщений (MessagesAdapter) с лайками: Класс адаптера MessagesAdapter был доработан для поддержки отметок «Нравится» и аватарок:

- Внутри адаптера заведено свойство likedIds: MutableSet<Int> – множество ID сообщений, которые пользователь «лайкнул». При повторном открытии приложения это не сохраняется (достаточно для демонстрации интерактивности в рамках сессии).
- Во время биндинга каждого элемента проверяем, есть ли его ID в likedIds, и устанавливаем иконку лайка соответственно: либо закрашенное сердце (ic_like_filled), либо контур (ic_like_border).
- Назначаем слушатель на ImageView лайка: при клике добавляем или удаляем ID сообщения из множества и обновляем иконку.

Код внутреннего ViewHolder с методом bind демонстрирует логику лайков:

```
inner class VH(private val binding:
ItemMessageBinding) :
RecyclerView.ViewHolder(binding.root) {
    fun bind(item: MessageEntity) {

binding.ivAvatar.setImageResource(R.mipmap.ic_launcher_round)
        binding.tvTitle.text = item.title
        binding.tvBody.text = item.body

        fun renderLike() {
            val liked = likedIds.contains(item.id)
            binding.ivLike.setImageResource(
                if (liked) R.drawable.ic_like_filled
else R.drawable.ic_like_border
            )
        }

        renderLike()

        binding.ivLike.setOnClickListener {
            if (likedIds.contains(item.id))
likedIds.remove(item.id) else likedIds.add(item.id)
            renderLike()
        }
    }
}
```

При первом вызове bind для сообщения вызывается renderLike(), которая ставит нужную картинку сердечка. Далее при нажатии на сердечко мы обновляем set likedIds и снова вызываем renderLike(), чтобы иконка поменялась. Таким образом, пользователь может тапать на сердечко, и оно будет переключаться (пустое/заполненное). Это решение локальное (не сохраняется в БД или на сервере), но демонстрирует навык работы с адаптером и пользовательскими событиями.

Адаптер наследуется от ListAdapter, поэтому применен стандартный DiffUtil для эффективности при обновлении списка (реализован как Diff object в классе). Также при создании элемента устанавливаем аватар ic_launcher_round.

WorkManager: фоновая синхронизация: Для автоматического обновления сообщений без участия пользователя настроен WorkManager. В App.onCreate() (класс Application, который мы назвали App и указали в манифесте) добавлен код планирования фоновой задачи:

```
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .build()

val request =
    PeriodicWorkRequestBuilder<SyncWorker>(15,
    TimeUnit.MINUTES)
        .setConstraints(constraints)
        .build()

WorkManager.getInstance(this).enqueueUniquePeriodicWork(
    "messages_sync",
    ExistingPeriodicWorkPolicy.KEEP,
    request
)
```

Здесь мы задаем условие только при наличии сети (NetworkType.CONNECTED) – чтобы не пытаться обновляться офлайн. WorkManager будет раз в 15 минут (минимально допустимый период для PeriodicWorkRequest) запускать нашу задачу SyncWorker. Используем enqueueUniquePeriodicWork с именем "messages_sync" и политикой KEEP, чтобы при повторных запусках приложения не дублировать задачи, а сохранить одну периодическую работу.

Класс SyncWorker: Наследует CoroutineWorker и выполняет обновление аналогично ручному:

```
class SyncWorker(
```

```

    appContext: Context,
    params: WorkerParameters
) : CoroutineWorker(appContext, params) {

    override suspend fun doWork(): Result {
        val app = applicationContext as App

        return try {
            app.messageRepository.refresh()
            SyncNotifier.show(applicationContext,
"Новые данные получены")
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

```

В `doWork()` мы достаём глобальный экземпляр приложения, получаем из него репозиторий (`Application` хранит его как `App.messageRepository`) и вызываем `refresh()` – тем самым загружаем новые данные и пишем их в базу. После успешного обновления сразу вызываем `SyncNotifier.show(...)` для отправки уведомления. `WorkManager`, благодаря указанным `Constraints`, сам убедится, что задача запускается только при наличии интернета, так что код внутри `try` уже может не беспокоиться о проверке сети. Если случится исключение (что маловероятно при правильных настройках, разве что сервер недоступен) – мы возвращаем `Result.retry()`, `WorkManager` попробует ещё через интервал. Таким образом, фоновые обновления настроены раз в ~15 минут. Уведомление покажет пользователю, что лента обновлена, даже если он не открывал приложение (например, телефон лежит в кармане).

Уведомления (Notification): Класс `SyncNotifier` – объект, инкапсулирующий создание и показ уведомления:

```

package com.example.messengerlab1.work.notify

import android.app.NotificationChannel
import android.app.NotificationManager

```

```
import android.content.Context
import android.os.Build
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import com.example.messengerlab1.R

object SyncNotifier {
    private const val CHANNEL_ID = "sync_channel"
    private const val NOTIFICATION_ID = 1

    fun show(context: Context, text: String) {
        ensureChannel(context)

        if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
            val granted =

context.checkSelfPermission(android.Manifest.permission.POST_NOTIFICATIONS) ==

android.content.pm.PackageManager.PERMISSION_GRANTED
            if (!granted) return
        }

        val notification =
NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.mipmap.ic_launcher)

.setContentTitle(context.getString(R.string.app_name)
) )
            .setContentText(text)
            .setAutoCancel(true)

.setPriority(NotificationCompat.PRIORITY_HIGH)
            .build()
```

```

NotificationManagerCompat.from(context).notify(NOTIFICATION_ID, notification)
    }

    private fun ensureChannel(context: Context) {
        if (Build.VERSION.SDK_INT <
Build.VERSION_CODES.O) return

        val nm =
context.getSystemService(NotificationManager::class.java)
        if (nm.getNotificationChannel(CHANNEL_ID) !=
null) return

        val channel = NotificationChannel(
            CHANNEL_ID,
            "Синхронизация",
            NotificationManager.IMPORTANCE_HIGH
        )
        nm.createNotificationChannel(channel)
    }
}

```

При первом показе уведомления для Android O+ мы создаем канал "sync_channel" (название "Синхронизация"). Для Android 13+ проверяем, выдал ли пользователь разрешение POST_NOTIFICATIONS – если нет, просто выходим (не будет ошибки, просто уведомление не покажется). Затем формируем NotificationCompat с иконкой приложения, заголовком (название нашего приложения) и текстом (передается параметр "Новые данные получены"). setAutoCancel(true) делает так, что по тапу уведомление исчезнет. NotificationManagerCompat.notify публикует уведомление с ID 1.

Этот код вызывается как из SyncWorker (фоновые обновления), так и при ручном refresh (через viewModel.syncSuccess.observe в ленте). Таким образом, любое успешное получение новых данных приводит к уведомлению. Само уведомление видно пользователю даже при свернутом

приложении (WorkManager работает в фоновом режиме). Разумеется, если пользователь не дал разрешение (на Android 13+) или отключил его – уведомления не будут приходить, что тоже учтено.

Permissions (разрешения): В манифест добавлены uses-permission: INTERNET (для сети), ACCESS_NETWORK_STATE (для проверки сети), POST_NOTIFICATIONS, READ_CONTACTS. На Android 13+ runtime-разрешение на уведомления обязательно – мы запросили его при старте приложения.

Прочие улучшения: Реализована реакция UI на системные события онлайн/офлайн: например, если пользователь запустил приложение без сети, а затем сеть появилась, нажатие на кнопку обновления (которую мы делаем активной при онлайн) сразу выполнит refresh.

Итог по этапу 4: После этого этапа приложение получило законченный вид и поведение:

- Интерфейс соответствует Material Design: удобная навигация, плавающая кнопка обновления, карточки с контентом, понятное отображение состояния.
- Добавлена интерактивность – пользователь может отмечать понравившиеся сообщения, это мгновенно отражается в UI (пусть и не сохраняется долговременно).
- Приложение не требует постоянного внимания – в фоне самостоятельно обновляет данные. Когда приходят новые сообщения (в нашем случае, JSONPlaceholder возвращает фиксированные 100 постов, но можно представить реализацию реального API с новыми данными), пользователь будет уведомлен.
- Все новые функции интегрированы без нарушения архитектуры: WorkManager вызывает код репозитория, не вмешиваясь в UI; уведомления отображаются через отдельный класс-утилиту, не засоряя бизнес-логику; адаптер остаётся только в слое представления и не хранит лишних данных (кроме likedIds для UI-состояния лайков).

Приложение сохранило принцип MVVM. Например, обработка лайка – чисто UI-функция (не влияющая на данные приложения), поэтому хранится внутри адаптера (вью-слой). Фоновая синхронизация – часть

модели/репозитория, и она никак не обращается напрямую к UI, а общается через обновление базы и уведомления.

5. Выводы

В ходе выполнения курсового проекта было разработано Android-приложение «Messenger» с использованием архитектуры MVVM и связанных технологий. Работа над проектом прошла через четыре этапа, каждый из которых добавлял новый уровень функциональности:

- На этапе 1 создан базовый каркас приложения с навигацией и тремя экранами, заложена структура для дальнейшего расширения.
- Этап 2 внедрил ViewModel и LiveData для хранения и восстановления состояния интерфейса, устранив проблему потери данных при поворотах и обеспечив реактивное обновление UI.
- Этап 3 подключил реальные данные: реализованы сетевой слой (Retrofit) и локальная база (Room), что позволило отображать список сообщений, синхронизировать их с сервером и работать офлайн.
- Этап 4 улучшил пользовательский опыт и архитектуру: применены Material Design компоненты для современного UI, добавлена интерактивность (лайки), фоновые обновления (WorkManager) и уведомления, расширены возможности приложения без ухудшения структуры кода.

Технически приложение соответствует заданным требованиям и демонстрирует правильное разделение ответственности:

- Activity/Fragment отвечают только за отображение и ввод, вся бизнес-логика (загрузка данных, обработка состояния) находится во ViewModel и ниже.
- ViewModel успешно изолирует UI от деталей данных. Он хранит минимально необходимые данные и использует репозиторий для всех операций с данными. Логи вызовов ViewModel (init/onCleared) подтвердили корректность жизненного цикла.
- Репозиторий реализован как единая точка доступа, скрывающая реализацию API/DB. Это повысило модульность – например, при необходимости заменить источник данных (другой API или добавить кэш) достаточно поменять код репозитория, UI и ViewModel трогать не потребуется.

- Room + LiveData обеспечивают устойчивое хранение и мгновенное уведомление об изменениях данных. Мы фактически получили реактивный поток данных: новые данные приходят – UI обновляется автоматически. Это избавило от необходимости вручную обновлять адаптер по результатам сетевых запросов – достаточно обновить базу.
- WorkManager позволил реализовать фоновые задачи согласно Android-рекомендациям, учитывая состояние устройства (сеть). Уведомления сделали работу приложения более прозрачной для пользователя.
- Material Design улучшил удобство интерфейса, а реализация likes и offline-индикатора повысила интерактивность и информативность.

Архитектура приложения получилась надежной и расширяемой. Добавление новых функций не привело к хаотичному росту связей – каждый слой остался со своей четкой зоной ответственности, что соответствует принципам SOLID и требованиям к профессиональным приложениям.

Можно заключить, что использование MVVM-подхода дало ожидаемые результаты: код стал более прозрачным, тестируемым и поддерживаемым. Например, бизнес-логику репозитория или ViewModel можно протестировать отдельно, подставив мок-версии API или DAO. UI-код минимизирован и не содержит сложных операций, поэтому меньше вероятность допустить утечки памяти или ошибки цикла жизни. Реализованное приложение может служить основой для дальнейшего развития: при необходимости несложно добавить, скажем, экран списка контактов (с собственной ViewModel и репозиторием) или функциональность отправки сообщений – архитектура готова к масштабированию.

Выполнение курсовой работы позволило получить ценный практический опыт в построении Android-приложений по современным стандартам. Все поставленные цели достигнуты: создано работоспособное приложение-мессенджер (в учебном формате) с соблюдением требований и грамотной архитектурой.